

Software Requirements Specification (SRS)

Employee Management System (EMS)

Document Version: 2.0

Status: Approved & Finalized

Architecture: Microservices with .NET 9 Web API Gateway & React Frontend

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document details the complete functional and non-functional requirements, data models, business rules, system architecture, and interface designs for the enterprise **Employee Management System (EMS)**. It serves as the primary contract between development, quality assurance, operations, and business stakeholders.

1.2 Scope

The Employee Management System is a centralized, cloud-ready, enterprise-grade human capital and workplace operations platform. The system coordinates identity, employee life cycle, organizational structure, compensation and revisions, time tracking and attendance, leave applications, support ticket resolution, and workplace reporting across distributed microservices connected through a unified reverse proxy API Gateway (YARP).

1.3 Intended Audience

- **Software Engineers & Architects:** For implementing, refining, and scaling microservices, gateway routing, and client applications.
- **QA Engineers:** For designing integration, end-to-end, and unit test suites across API contracts.
- **System & DevOps Administrators:** For provisioning databases, network policies, and container runtimes.
- **Human Resources & Business Leadership:** For validating business rules, approval workflows, and audit capabilities.

1.4 Definitions and Abbreviations

Term	Definition
EMS	Employee Management System

Term	Definition
YARP	Yet Another Reverse Proxy (.NET Core reverse proxy toolkit)
JWT	JSON Web Token (RFC 7519)
RBAC	Role-Based Access Control
DTO	Data Transfer Object
RFC 7807	Problem Details for HTTP APIs (Standardized error response payload)
UUID	Universally Unique Identifier (RFC 4122)
PNA	Private Network Access (Browser security specification)

1.5 References

- RFC 7519: JSON Web Token (JWT)
 - RFC 7807: Problem Details for HTTP APIs
 - Microsoft YARP (Yet Another Reverse Proxy) Documentation
 - OpenAPI / Swagger 3.0 Specifications for EMS Microservices
-

2. Overall Description

2.1 Product Perspective

EMS operates as a distributed system composed of independent backend domain services fronted by a centralized YARP API Gateway running on port 5000. The frontend is a single-page React application that communicates strictly through the gateway. Direct communication to individual microservice ports from the client layer is forbidden.

2.2 Product Features

- **Centralized Authentication:** Secure JWT-based credential validation and role-claims propagation.
- **Employee Directory & Lifecycle:** Comprehensive personnel records, search, role assignments, and status tracking.
- **Department Administration:** Organizational unit hierarchy, descriptions, and workforce headcount mapping.
- **Attendance & Time Tracking:** Real-time digital clock, daily check-in, checkout calculation, and historical logs.
- **Leave Lifecycle:** Application submission, start/end boundary validation, balance consumption, and manager approval/rejection.
- **Compensation & Payroll:** Base salary tracking, earnings and deductions component breakdowns, bonus grants, overtime authorization, monthly payroll generation, and itemized payslip generation.
- **Workplace Support:** Issue ticketing, multi-priority categorization, staff assignment, and lifecycle resolution.

2.3 User Types and Roles

1. **Admin:** Full administrative authority over users, employee records, organizational departments, salary configurations, payroll execution, and ticket assignment.
2. **HR (Human Resources):** Authority over employee onboarding, department staffing, salary revisions, bonuses, overtime review, and leave auditing.

3. **Manager:** Departmental supervisor authority over team attendance, leave application approvals/rejections, and overtime recommendations.
4. **Employee:** Standard self-service user with access to their own profile, personal attendance check-in/out, leave applications, compensation details, payslips, and support ticket creation.
5. **Support Staff:** Technical and workplace operations personnel responsible for triage, handling, and resolving submitted tickets.

2.4 Operating Environment

- **Client Tier:** Modern standards-compliant web browsers (Chromium, Firefox, Safari, Edge).
- **Frontend Server:** Node.js / Vite SPA runtime listening on port 3000.
- **Gateway Tier:** .NET 9 YARP API Gateway listening on `http://localhost:5000`.
- **Service Tier:** .NET 9 ASP.NET Core Web API microservices.
- **Persistence Tier:** Relational SQL Database (PostgreSQL / SQL Server) with isolated schemas or databases per service.

2.5 System Constraints

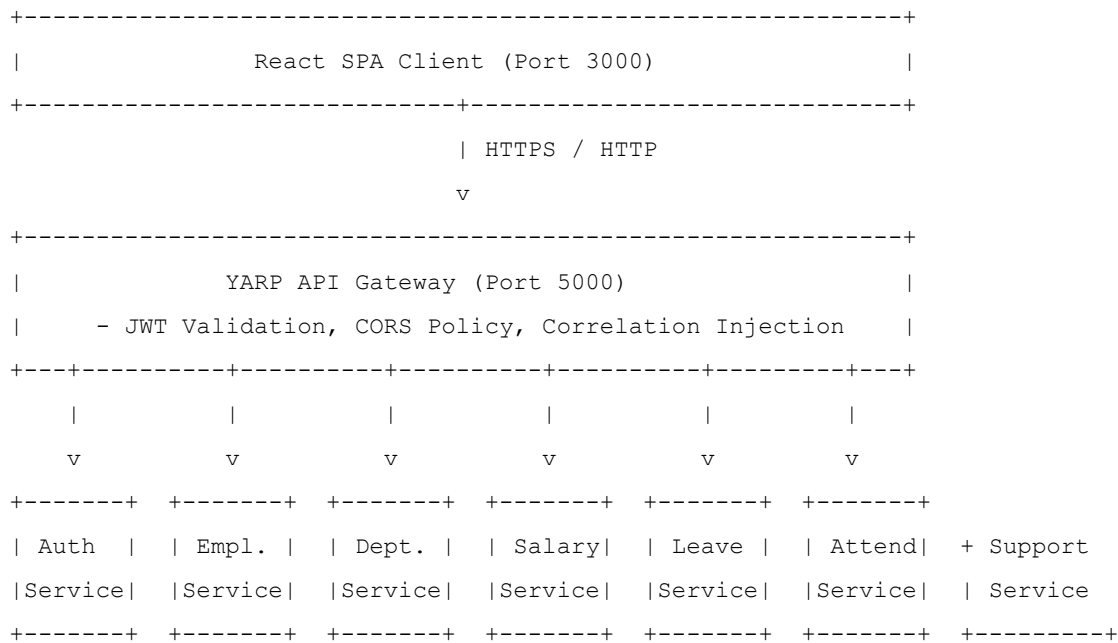
- All client requests must be routed via `http://localhost:5000/api/v1/*`.
- Client credentials must be supplied via HTTP Header `Authorization: Bearer <token>`.
- Client requests must include `X-Correlation-ID` for distributed request tracing.
- Financial records (Salary revisions and generated payroll items) are immutable once finalized.

2.6 Assumptions and Dependencies

- Network connectivity between the client browser, API Gateway, and microservice hosts.
- Clock synchronization across all service containers via NTP.
- User accounts are provisioned by an administrator; self-service registration is restricted.

3. System Architecture

3.1 Architecture Diagram



3.2 Frontend Architecture

Built with React, TypeScript, and Tailwind CSS.

- **State & Context:** Modular React Context providers (`AuthContext`, `GatewayConfigContext`, `ToastContext`).
- **HTTP Abstraction:** Uniform `ApiClient` wrapper automatically injecting auth tokens, correlation headers, and parsing RFC 7807 error responses.
- **Route Guarding:** Role-based path access control matching Wireframe screens.

3.3 Backend Architecture

Composed of 7 specialized services:

1. `AuthService`: Authentication, password hashing, and token issuance.
2. `EmployeeService`: Employee entity maintenance, profile retrieval, and search.
3. `DepartmentService`: Department categorization and directory management.
4. `SalaryService`: Salaries, components, revisions, bonuses, overtime, payrolls, and payslips.
5. `LeaveService`: Leave request submissions, history, and approval/rejection operations.
6. `AttendanceService`: Check-in, check-out, and daily logs.
7. `SupportService`: Ticket submission, assignment, and status workflows.

3.4 Database Architecture

Database-per-service pattern ensuring loose coupling. Each microservice manages its own tables, migrations, and transactional boundaries.

3.5 Authentication and Security Architecture

Stateless authentication using cryptographically signed JSON Web Tokens (HMAC-SHA256 or RSA-256). Claims contain `sub` (User ID), `role`, `email`, and `employeeId`.

4. Functional Requirements

4.1 Authentication Service (AuthService)

- **FR-AUT-001:** The system shall authenticate user credentials against stored secure password hashes.
- **FR-AUT-002:** The system shall issue a signed JWT upon successful authentication containing identity and role claims.
- **FR-AUT-003:** The system shall return standard HTTP 401 Unauthorized for invalid credentials.
- **FR-AUT-004:** The system shall support administrative account provisioning via registration endpoints.

4.2 Employee Management Service (EmployeeService)

- **FR-EMP-001:** The system shall list all employee records for authorized roles (Admin, HR).
- **FR-EMP-002:** The system shall create new employee records linked to user identities.
- **FR-EMP-003:** The system shall support updating employee details (name, email, phone, joining date, department, role, status).
- **FR-EMP-004:** The system shall provide single-employee lookup by UUID
(`/api/v1/employees/{employeeId}`).
- **FR-EMP-005:** The system shall allow authenticated employees to view their own profile via
(`/api/v1/employees/me`).
- **FR-EMP-006:** The system shall support fuzzy substring searching across names, roles, and emails.
- **FR-EMP-007:** The system shall support soft/hard deletion of employee records by Admin.

4.3 Department Management Service (DepartmentService)

- **FR-DEP-001:** The system shall provide a directory of all corporate departments.
- **FR-DEP-002:** The system shall allow authorized roles (Admin, HR) to create new departments.
- **FR-DEP-003:** The system shall allow updating department names and descriptions.
- **FR-DEP-004:** The system shall prohibit department deletion if active employees are assigned to it (BR-DEP-002).

4.4 Project Management

- **FR-PRJ-001:** The system shall support future project and workload allocations tied to departments and assigned employees.
- **FR-PRJ-002:** The system interface shall accommodate project staffing views and department-level assignments.

4.5 Salary & Payroll Service (SalaryService)

- **FR-SAL-001:** The system shall maintain employee base salary records and effective dates.
- **FR-SAL-002:** The system shall allow adding and editing salary components (allowances, earnings, deductions).
- **FR-SAL-003:** The system shall maintain an immutable, audited history of salary revisions with change reasons.
- **FR-SAL-004:** The system shall record performance and ad-hoc bonuses with effective dates.
- **FR-SAL-005:** The system shall accept overtime submissions from employees (hours, date, rate).
- **FR-SAL-006:** The system shall enable managers/HR to approve or reject overtime requests.
- **FR-SAL-007:** The system shall execute monthly batch payroll calculations for a given month and year.
- **FR-SAL-008:** The system shall generate itemized payslips from finalized payroll records.

4.6 Leave Management Service (LeaveService)

- **FR-LEV-001:** The system shall allow employees to apply for leave specifying leave type, start date, end date, and reason.
- **FR-LEV-002:** The system shall retrieve employee-specific leave history.
- **FR-LEV-003:** The system shall enable supervisors to approve leave requests with reviewer recording.
- **FR-LEV-004:** The system shall enable supervisors to reject leave requests with reviewer recording.
- **FR-LEV-005:** The system shall prevent users from approving their own leave requests (BR-GEN-005).

4.7 Attendance Management Service (AttendanceService)

- **FR-ATT-001:** The system shall record daily employee check-in timestamps.
- **FR-ATT-002:** The system shall prevent multiple open check-ins for the same employee on the same date.
- **FR-ATT-003:** The system shall record checkout timestamps and calculate total daily working duration.
- **FR-ATT-004:** The system shall retrieve personal attendance histories and organization-wide attendance registers.

4.8 Support Service (SupportService)

- **FR-SUP-001:** The system shall allow employees to submit support tickets with title, description, and priority (Low, Medium, High).
- **FR-SUP-002:** The system shall list all tickets for support staff and personal tickets for standard employees.
- **FR-SUP-003:** The system shall allow support staff to assign tickets to specific operators.
- **FR-SUP-004:** The system shall support updating ticket statuses (Open, Assigned, InProgress, Resolved, Closed).

5. User Roles and Permissions

5.1 Roles Definition

- **Admin:** System owner with unrestricted read/write permissions.

- **HR:** Human resources manager with full workforce and compensation management capabilities.
- **Manager:** Operational supervisor with team oversight and approval permissions.
- **Employee:** Individual contributor with self-service capabilities.
- **Support:** Operations personnel with ticket triage and resolution permissions.

5.2 Permissions Matrix

Functional Module	Action / Endpoint	Admin	HR	Manager	Employee	Support
Authentication	Login (/auth/login)	Yes	Yes	Yes	Yes	Yes
Authentication	Register (/auth/register)	Yes	No	No	No	No
Employees	View Directory (GET /employees)	Yes	Yes	Yes	Self	Read
Employees	Create / Edit / Delete	Yes	Yes	No	No	No
Departments	View (GET /departments)	Yes	Yes	Yes	Yes	Yes
Departments	Create / Edit / Delete	Yes	Yes	No	No	No
Attendance	Check-in / Check-out	Yes	Yes	Yes	Yes	Yes
Attendance	View Team Attendance	Yes	Yes	Yes	No	No
Leaves	Apply (POST /leaves)	Yes	Yes	Yes	Yes	Yes
Leaves	Approve / Reject	Yes	Yes	Yes	No	No
Salary	View Own Salary / Payslip	Yes	Yes	Yes	Yes	Yes
Salary	Configure Salary / Revisions	Yes	Yes	No	No	No
Salary	Overtime Request	Yes	Yes	Yes	Yes	Yes
Salary	Overtime Approve / Reject	Yes	Yes	Yes	No	No
Payroll	Generate Batch Payroll	Yes	Yes	No	No	No
Support	Create Ticket	Yes	Yes	Yes	Yes	Yes
Support	Assign / Update Status	Yes	No	No	No	Yes

6. Database Requirements

6.1 Database Engine

Relational DBMS (PostgreSQL 15+ / Microsoft SQL Server 2022). Tables enforce primary keys, foreign keys, uniqueness constraints, and indexation on frequently queried identifiers.

6.2 Entities & Schemas

1. AuthService

- **Users:** UserId (PK, UUID), Username (Unique), Email (Unique), PasswordHash, Role, CreatedAt, UpdatedAt.

2. EmployeeService

- **Employees:** EmployeeId (PK, UUID), UserId (FK, UUID), FirstName, LastName, Email (Unique), Phone, DateOfJoining, DepartmentId (FK, UUID), Role, EmploymentStatus, CreatedAt, UpdatedAt.

3. DepartmentService

- **Departments:** DepartmentId (PK, UUID), DepartmentName (Unique), Description, CreatedAt, UpdatedAt.

4. SalaryService

- **EmployeeSalaries:** EmployeeSalaryId (PK, UUID), EmployeeId (Unique, UUID), BasicSalary, EffectiveFrom, CreatedAt, UpdatedAt.
- **SalaryComponents:** SalaryComponentId (PK, UUID), EmployeeId (FK, UUID), ComponentName, Amount, ComponentType (Earning/Deduction), CreatedAt, UpdatedAt.
- **SalaryRevisions:** SalaryRevisionId (PK, UUID), EmployeeId (FK, UUID), PreviousSalary, RevisedSalary, RevisionDate, Reason, CreatedAt, UpdatedAt.
- **Bonuses:** BonusId (PK, UUID), EmployeeId (FK, UUID), BonusType, Amount, BonusDate, Reason, CreatedAt, UpdatedAt.
- **Overtimes:** OvertimeId (PK, UUID), EmployeeId (FK, UUID), OvertimeDate, Hours, Rate, Amount, Status (Pending/Approved/Rejected), ApprovedBy, ApprovedAt, CreatedAt, UpdatedAt.
- **Payrolls:** PayrollId (PK, UUID), PayrollMonth (Int), PayrollYear (Int), GeneratedAt, Status, CreatedAt, UpdatedAt.
- **PayrollItems:** PayrollItemId (PK, UUID), PayrollId (FK, UUID), EmployeeId (FK, UUID), BasicSalary, TotalComponents, TotalBonus, OvertimeAmount, TotalDeductions, NetSalary, CreatedAt, UpdatedAt.
- **Payslips:** PayslipId (PK, UUID), PayrollId (FK, UUID), EmployeeId (FK, UUID), PayslipNumber (Unique), PayslipDate, BasicSalary, TotalComponents, TotalBonus, OvertimeAmount, TotalDeductions, NetSalary, CreatedAt, UpdatedAt.

5. LeaveService

- **Leaves:** LeaveId (PK, UUID), EmployeeId (FK, UUID), LeaveType, StartDate, EndDate, Reason, Status (Pending/Approved/Rejected), ApprovedBy, ApprovedAt, AppliedAt, CreatedAt, UpdatedAt.

6. AttendanceService

- **Attendances:** AttendanceId (PK, UUID), EmployeeId (FK, UUID), AttendanceDate (Date), CheckInTime, CheckOutTime (Nullable), Status (CheckedIn/CheckedOut), CreatedAt, UpdatedAt.

7. SupportService

- **SupportTickets:** TicketId (PK, UUID), EmployeeId (FK, UUID), Title, Description, Priority (Low/Medium/High), Status (Open/Assigned/InProgress/Resolved/Closed), AssignedTo (Nullable, UUID), CreatedAt, UpdatedAt, ResolvedAt (Nullable).

6.3 Normalization

All service schemas adhere strictly to Third Normal Form (3NF). Data duplication across service boundaries is restricted to primary keys/foreign identifiers.

7. Non-Functional Requirements

7.1 Performance

- **Response Time:** Gateway response time < 200ms for 95% of queries under normal load.
- **Concurrent Users:** System sustains at least 500 concurrent sessions without performance degradation.

7.2 Security

- TLS 1.3 encryption across all public network transport.
- Cryptographically secure hashing (BCrypt / Argon2 / PBKDF2) for stored passwords.
- Strict JWT expiry and claims verification on every non-public endpoint.

7.3 Reliability & Availability

- High availability target of 99.9% uptime during operational business hours.
- Automatic container recovery and health check probes on gateway routes.

7.4 Scalability

- Microservices scale horizontally independent of other domains.
- Stateless API gateways allowing load distribution across multiple instances.

7.5 Maintainability

- Consistent RESTful API conventions across all services.
- Detailed logging with `X-Correlation-ID` for end-to-end request tracing.

7.6 Usability

- Modern responsive web interface adhering to WCAG 2.1 Level AA standards.
- Intuitive workflow validation and descriptive error messages.

7.7 Data Integrity

- Transactional ACID compliance on all compensation and payroll mutations.
 - Foreign key constraints preventing orphaned child records.
-

8. Security Requirements

8.1 Authentication

- Endpoint protection via standard `Authorization: Bearer <JWT>` headers.
- Automatic session termination upon token expiration.

8.2 Authorization & RBAC

- Role-based policy enforcement evaluated at both the API Gateway and microservice controller levels.
- Unauthorized access attempts return HTTP 403 Forbidden with RFC 7807 problem details.

8.3 Password Security

- Passwords must meet complexity requirements (minimum 8 characters, uppercase, lowercase, numeric, and special characters).
- Plaintext passwords are never stored or logged in system traces.

8.4 Input Validation

- Server-side DTO validation using data annotations or FluentValidation.
 - Automatic sanitization of inputs preventing SQL Injection and Cross-Site Scripting (XSS).
-

9. API Requirements

9.1 API Overview

All communication uses JSON over HTTP/1.1 or HTTP/2. Base URL: `http://localhost:5000/api/v1/`.

9.2 Endpoint Summary

Service	Method	Route	Description
Auth	POST	<code>/api/v1/auth/login</code>	Authenticate user and issue JWT
Auth	POST	<code>/api/v1/auth/register</code>	Register new user account
Employee	GET	<code>/api/v1/employees</code>	List all employees
Employee	POST	<code>/api/v1/employees</code>	Create employee record
Employee	GET	<code>/api/v1/employees/{id}</code>	Get employee by ID
Employee	PUT	<code>/api/v1/employees/{id}</code>	Update employee
Employee	DELETE	<code>/api/v1/employees/{id}</code>	Delete employee
Employee	GET	<code>/api/v1/employees/search</code>	Search employees by term

Service	Method	Route	Description
Employee	GET	/api/v1/employees/me	Get authenticated employee profile
Department	GET	/api/v1/departments	List all departments
Department	POST	/api/v1/departments	Create department
Department	GET	/api/v1/departments/{id}	Get department by ID
Department	PUT	/api/v1/departments/{id}	Update department
Department	DELETE	/api/v1/departments/{id}	Delete department
Salary	POST	/api/v1/salaries	Create base salary
Salary	GET	/api/v1/salaries/{empId}	Get employee salary
Salary	PUT	/api/v1/salaries/{empId}	Update employee salary
Salary	POST	/api/v1/salaries/{empId}/components	Add salary component
Salary	GET	/api/v1/salaries/{empId}/components	Get salary components
Salary	PUT	/api/v1/salary-components/{id}	Update salary component
Salary	POST	/api/v1/salaries/{empId}/revisions	Record salary revision
Salary	GET	/api/v1/salaries/{empId}/revisions	List salary revisions
Salary	POST	/api/v1/salaries/{empId}/bonuses	Grant bonus
Salary	GET	/api/v1/salaries/{empId}/bonuses	List bonuses
Salary	PUT	/api/v1/bonuses/{id}	Update bonus
Salary	POST	/api/v1/overtime	Submit overtime request
Salary	GET	/api/v1/overtime/{empId}	Get employee overtime records
Salary	PUT	/api/v1/overtime/{id}/approve	Approve overtime
Salary	PUT	/api/v1/overtime/{id}/reject	Reject overtime
Salary	POST	/api/v1/payroll	Generate monthly payroll
Salary	GET	/api/v1/payroll	List generated payrolls
Salary	GET	/api/v1/payroll/{id}	Get payroll by ID
Salary	GET	/api/v1/payroll/{id}/items	Get items in payroll
Salary	POST	/api/v1/payslips/{payId}/{empId}	Generate payslip
Salary	GET	/api/v1/payslips	List payslips
Salary	GET	/api/v1/payslips/{id}	Get payslip details
Leave	POST	/api/v1/leaves	Apply for leave
Leave	GET	/api/v1/leaves/{id}	Get leave details
Leave	GET	/api/v1/leaves/employee/{empId}	List employee leaves
Leave	GET	/api/v1/leaves/{empId}/history	Get employee leave history
Leave	PUT	/api/v1/leaves/{id}/approve	Approve leave
Leave	PUT	/api/v1/leaves/{id}/reject	Reject leave
Attendance	POST	/api/v1/attendance/check-in	Record daily check-in
Attendance	PUT	/api/v1/attendance/{id}/check-out	Record daily check-out
Attendance	GET	/api/v1/attendance/{id}	Get attendance record
Attendance	GET	/api/v1/attendance/employee/{empId}	List employee attendance logs
Attendance	GET	/api/v1/attendance	List all attendance logs
Support	POST	/api/v1/support-tickets	Create support ticket

Service	Method	Route	Description
Support	GET	/api/v1/support-tickets	List all support tickets
Support	GET	/api/v1/support-tickets/{id}	Get ticket details
Support	PUT	/api/v1/support-tickets/{id}	Update ticket status / assignee
Support	GET	/api/v1/support-tickets/employee/{id}	List employee tickets

9.3 Standard Response & Error Formats

Successful responses return JSON payloads with standard HTTP status codes (200 OK, 201 Created, 204 No Content). Errors return RFC 7807 ProblemDetails:

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "Validation Error",
  "status": 400,
  "detail": "Start date cannot be after end date.",
  "instance": "/api/v1/leaves",
  "traceId": "00-84a2f8-01"
}
```

10. Business Rules

10.1 Employee Rules

- **BR-EMP-001:** Every employee must possess a valid, unique corporate email address.
- **BR-EMP-002:** Date of joining cannot be in the future.
- **BR-EMP-003:** Deletion of an employee marks related history as archived.

10.2 Department Rules

- **BR-DEP-001:** Department names must be unique across the organization.
- **BR-DEP-002:** A department cannot be deleted if active employees are currently assigned to it.

10.3 Salary & Payroll Rules

- **BR-SAL-001:** Base salary must be a positive non-zero currency value.
- **BR-SAL-002:** Salary revisions are strictly append-only and audited with the author and reason.
- **BR-SAL-003:** Overtime compensation = Hours * Rate. Overtime must be approved before payroll inclusion.
- **BR-SAL-004:** Generated payroll for a closed cycle cannot be recalculated without an administrative unlock.

10.4 Leave Rules

- **BR-LEV-001:** Leave `StartDate` must precede or equal `EndDate`.
- **BR-LEV-002:** An employee cannot apply for concurrent overlapping leave periods.
- **BR-LEV-003:** An employee cannot approve or reject their own leave application.

10.5 Attendance Rules

- **BR-ATT-001:** Only one check-in is permitted per employee per calendar date.
- **BR-ATT-002:** Check-out timestamp must be chronological after the check-in timestamp.

10.6 Project Rules

- **BR-PRJ-001:** Team members assigned to projects must belong to active departments.
- **BR-PRJ-002:** Project allocations reflect live operational capacity without exceeding standard weekly thresholds.

10.7 Support Rules

- **BR-SUP-001:** Tickets transition through: `Open` -> `Assigned` -> `InProgress` -> `Resolved` -> `Closed`.
 - **BR-SUP-002:** Only staff with `Support` or `Admin` roles may take ownership of assigned tickets.
-

11. UI Requirements

The frontend implementation conforms to the 12 wireframe specifications:

1. **Login Screen:** Clean two-panel layout with branded hero, metric highlights, credentials input, role preview selector, and diagnostic network indicators.
2. **Dashboard:** Consolidated stat cards (Headcount, Attendance, Pending Leaves, Monthly Payroll), department distribution visual bars, pending leave review widget, ticket counts, and quick actions.
3. **Employee Directory & Profile:** Searchable and filterable employee table, add/edit modal forms, view details drawer, and delete confirmation dialogs.
4. **Department Management:** Department catalog displaying live assigned employee counts, create modal, edit modal, and BR-DEP-002 delete restrictions.
5. **Project / Allocation Screen:** Integrated staffing view showing department allocations and workload distributions.
6. **Salary & Payroll (Employee View):** Base compensation overview, allowances/deductions breakdown, overtime request drawer, salary revision audit history, and issued payslip viewing.
7. **Salary & Payroll (Admin View):** Base salary assignment, component builder, revision recorder, bonus grant modal, overtime approval queue, batch payroll generator, and itemized payslip generator.
8. **Leave Management:** Interactive leave request form, circular balance indicators, personal leave history, and supervisory review queue with one-click approve/reject.
9. **Attendance Management:** Digital real-time clock, dynamic Check-In / Check-Out buttons, daily/weekly hours calculation, personal history log, and team register.

10. **Support Screens:** Ticket submission form with priority tags, ticket queue counters, assignment selector, and status workflow transitions.
 11. **User Profile Screen:** Personal profile card, department info, employment timeline, and restricted self-service profile editor.
-

12. Error Handling

12.1 Validation Errors (HTTP 400)

Returned when input parameters violate field length, nullability, or business rule constraints. The UI highlights invalid fields inline.

12.2 Authentication Errors (HTTP 401)

Returned when tokens are missing, malformed, or expired. The client tears down the local session and redirects to `/login`.

12.3 Authorization Errors (HTTP 403)

Returned when an authenticated user attempts an operation outside their role. Handled via in-app access denial toasts without terminating the session.

12.4 Not Found Errors (HTTP 404)

Returned when a requested resource identifier does not exist. Handled via friendly empty states or a 404 screen.

12.5 Database & Conflict Errors (HTTP 409)

Returned when unique constraints (e.g. duplicate email, simultaneous check-in) are violated. Descriptive conflict messages are surfaced to the user.

12.6 Server Errors (HTTP 500)

Unhandled internal exceptions return standardized RFC 7807 payloads with `TraceId` for server log correlation.

13. Testing Requirements

13.1 Unit Testing

- Test calculation logic for overtime amounts (`Hours * Rate`).
- Test validation logic for leave date bounds (`StartDate <= EndDate`).

- Test password strength validator and token parsing routines.

13.2 Integration Testing

- Test database repository CRUD operations under active transactions.
- Test cross-service identity propagation via `Authorization` header.

13.3 API Testing

- Validate all 50+ endpoints against OpenAPI/Swagger schemas.
- Verify status codes and RFC 7807 error formats on invalid inputs.

13.4 Functional & Workflow Testing

- End-to-end leave lifecycle: Submit -> Pending -> Approve -> Approved.
- End-to-end overtime lifecycle: Submit -> Pending -> Approve -> Included in Payroll.
- End-to-end payroll generation and itemized payslip generation.

13.5 Security Testing

- Verify endpoint protection: non-authenticated requests must receive 401.
- Verify role restrictions: non-admin roles calling admin endpoints must receive 403.
- Verify SQL injection resistance and XSS sanitization.

14. Appendices

14.1 Entity Relationship Diagram (ERD Overview)

```
[Users] 1 --- 1 [Employees]
      |
      +--- N [Attendances]
      +--- N [Leaves]
      +--- N [SupportTickets]
      +--- 1 [EmployeeSalaries] --- N [SalaryComponents]
      |                               --- N [SalaryRevisions]
      |                               --- N [Bonuses]
      |                               --- N [Overtimes]
      +--- N [PayrollItems] <--- 1 [Payrolls]
      +--- N [Payslips]      <--- 1 [Payrolls]
[Departments] 1 --- N [Employees]
```

14.2 Role & Operational Sequence (Leave Application)

1. **Employee** submits `POST /api/v1/leaves` with details.
2. **LeaveService** validates dates (BR-LEV-001) and saves with status `Pending`.
3. **Manager** views pending requests and submits `PUT /api/v1/leaves/{id}/approve` with Manager UUID.
4. **LeaveService** validates non-self approval (BR-LEV-003) and updates status to `Approved`.

14.3 Active Microservices Registry

Service Name	Port (Internal)	Gateway Route Prefix	Primary Entity
AuthService	5001	<code>/api/v1/auth/*</code>	User
EmployeeService	5002	<code>/api/v1/employees/*</code>	Employee
DepartmentService	5003	<code>/api/v1/departments/*</code>	Department
SalaryService	5004	<code>/api/v1/salaries/*, /api/v1/payroll/*, /api/v1/overtime/*, /api/v1/payslips/*</code>	Salary, Payroll
LeaveService	5005	<code>/api/v1/leaves/*</code>	Leave
AttendanceService	5006	<code>/api/v1/attendance/*</code>	Attendance
SupportService	5007	<code>/api/v1/support-tickets/*</code>	SupportTicket

14.4 Glossary

- **YARP**: Reverse proxy layer standardizing routing, logging, and security.
- **ProblemDetails**: Standardized error model ensuring frontend can parse error specifics consistently.
- **Base Salary**: Fixed monthly compensation before allowances, deductions, and bonuses.
- **Net Salary**: Final calculated take-home pay: `Base + Earnings + Bonuses + Overtime - Deductions`.

